Notifications Feature

Relevant source files

- `Frontend/STUDER/src/app/features/notifications/notification.model.ts`
- `Frontend/STUDER/src/app/features/notifications/notification.service.ts`
- `Frontend/STUDER/src/app/features/notifications/strategy/default-notification.strategy.ts`
- `Frontend/STUDER/src/app/features/notifications/strategy/discussion-notification.strategy.ts`
- `Frontend/STUDER/src/app/features/notifications/strategy/message-notification.strategy.ts`
- `Frontend/STUDER/src/app/features/notifications/strategy/notification-strategy.factory.ts`
- `Frontend/STUDER/src/app/features/notifications/strategy/notification.strategy.ts`
- `Frontend/STUDER/src/app/features/notifications/strategy/system-notification.strategy.ts`
- `Frontend/STUDER/src/app/features/notifications/strategy/user-notification.strategy.ts`

The Notifications feature in STUDER provides a reactive system for alerting users about platform activities. It utilizes a **Strategy Pattern** to handle different notification types (discussions, messages, user interactions, etc.) uniformly while allowing type-specific routing and iconography.

Notification Data Models

The system is built around the `NotificationResponseDTO`, which contains the payload for a single notification, and a `LinkedType` enum that determines the notification's context and behavior.

Core Types and Interfaces

Interface / Type	Purpose
`LinkedType`	Defines the source of the notification: `COURSE`, `DISCUSSION`, `ACTIVITY`, `MESSAGE`, `USER`, `SYSTEM`, or `EVENT` `Frontend/STUDER/src/app/features/notifications/notification.model.ts 1-8`
`NotificationResponseDTO`	The main data structure containing the `title`, `message`, `type`, and an optional `linkedId` used for navigation `Frontend/STUDER/src/app/features/notifications/notification.model.ts 10-18`
`NotificationPageResponseDTO`	A paginated wrapper for a list of notifications `Frontend/STUDER/src/app/features/notifications/notification.model.ts 20-25`

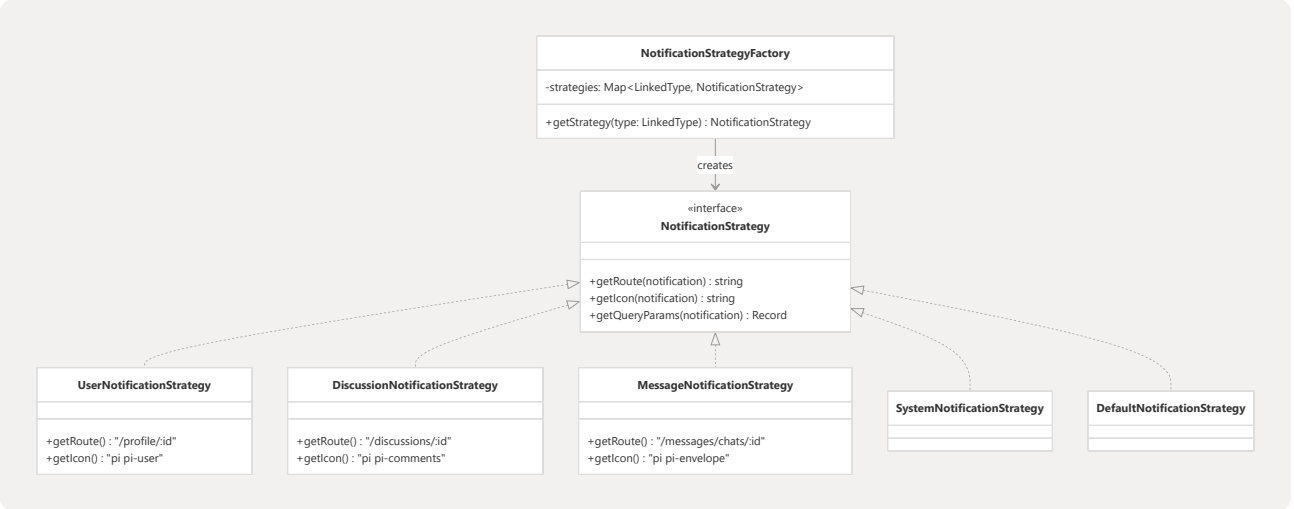
Sources: `Frontend/STUDER/src/app/features/notifications/notification.model.ts 1-31`

Strategy Pattern Implementation

To avoid complex conditional logic (switch/if-else) when rendering notifications, STUDER uses the Strategy Pattern. Each notification type implements the `NotificationStrategy` interface.

Strategy Architecture

The `NotificationStrategyFactory` initializes a map of strategies using the Angular `Injector` and returns the appropriate implementation based on the `LinkedType`.



Concrete Strategies

Strategy	LinkedType	Behavior
`UserNotificationStrategy`	`USER`	Routes to `/profile/${linkedId}`. Uses `pi-user` icon `Frontend/STUDER/src/app/features/notifications/strategy/user-notification.strategy.ts` 9-15
`DiscussionNotificationStrategy`	`DISCUSSION`	Routes to `/discussions/${linkedId}`. Uses `pi-comments` icon `Frontend/STUDER/src/app/features/notifications/strategy/discussion-notification.strategy.ts` 9-15
`MessageNotificationStrategy`	`MESSAGE`	Routes to `/messages/chats/${linkedId}`. Uses `pi-envelope` icon `Frontend/STUDER/src/app/features/notifications/strategy/message-notification.strategy.ts` 9-15
`SystemNotificationStrategy`	`SYSTEM`	Routes to `/home`. Uses `pi-info-circle` icon `Frontend/STUDER/src/app/features/notifications/strategy/system-notification.strategy.ts` 9-15
`DefaultNotificationStrategy`	**Fallback**	Routes to `/home`. Uses `pi-bell` icon `Frontend/STUDER/src/app/features/notifications/strategy/default-notification.strategy.ts` 9-15

Sources:

`Frontend/STUDER/src/app/features/notifications/strategy/notification.strategy.ts` 3-7

`Frontend/STUDER/src/app/features/notifications/strategy/notification-strategy.factory.ts` 13-27

`Frontend/STUDER/src/app/features/notifications/strategy/user-notification.strategy.ts` 8-20

`Frontend/STUDER/src/app/features/notifications/strategy/discussion-notification.strategy.ts` 8-20

`Frontend/STUDER/src/app/features/notifications/strategy/message-notification.strategy.ts` 8-20

`Frontend/STUDER/src/app/features/notifications/strategy/system-notification.strategy.ts` 8-20

`Frontend/STUDER/src/app/features/notifications/strategy/default-notification.strategy.ts` 8-20

Notification Service

The `NotificationService` acts as the orchestrator for fetching notifications, managing the unread count state, and interacting with the strategy factory.

State Management

The service maintains a `BehaviorSubject` called `unreadCountSubject` `Frontend/STUDER/src/app/features/notifications/notification.service.ts 15` This allows components (like the Header badge) to reactively display the number of pending notifications without redundant API calls.

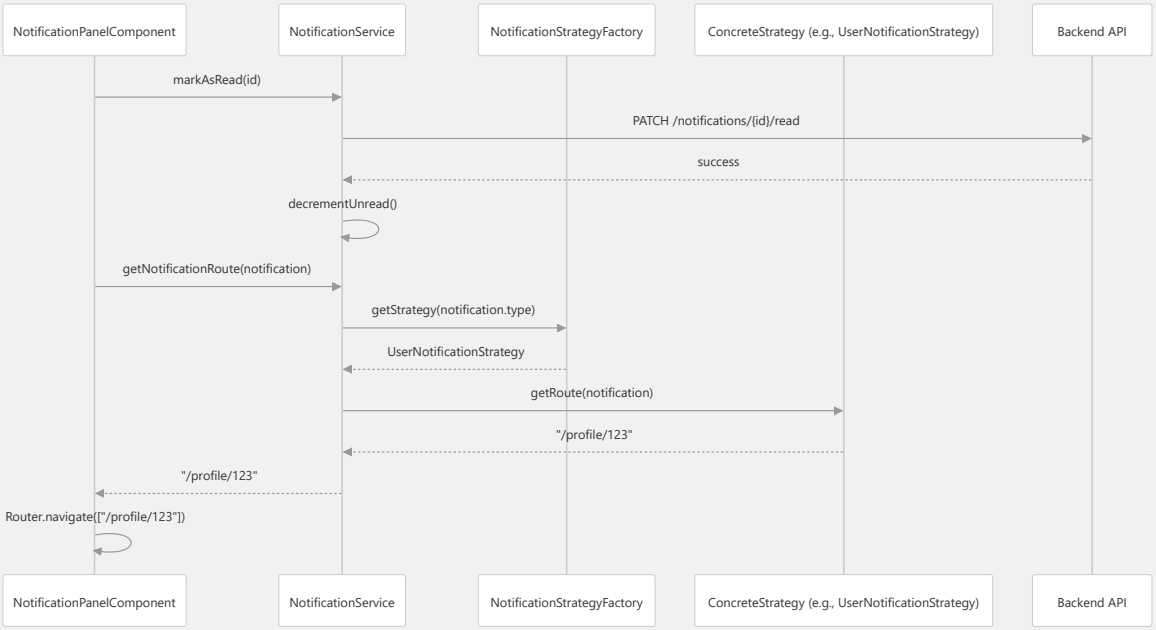
Key Functions

- `getNotifications`: Fetches paginated notifications from the backend. Supports filtering by `type`, `read` status, and `lastDays`
  `Frontend/STUDER/src/app/features/notifications/notification.service.ts 23-45`
- `markAsRead`: Sends a `PATCH` request to update the notification status
  `Frontend/STUDER/src/app/features/notifications/notification.service.ts 47-52`
- `loadUnreadCount`: Queries the API for unread notifications and updates the internal `unreadCountSubject`
  `Frontend/STUDER/src/app/features/notifications/notification.service.ts 54-59`
- `getNotificationRoute`: Delegates to the `NotificationStrategyFactory` to determine where the user should be redirected when clicking a notification
  `Frontend/STUDER/src/app/features/notifications/notification.service.ts 68-71`

Sources: `Frontend/STUDER/src/app/features/notifications/notification.service.ts 14-82`

Data Flow: Handling a Notification Interaction

This diagram illustrates how a user interaction with a notification item is processed through the service and strategy layers.



Sources: `Frontend/STUDER/src/app/features/notifications/notification.service.ts` `47-52`

`Frontend/STUDER/src/app/features/notifications/notification.service.ts` `61-71`

`Frontend/STUDER/src/app/features/notifications/strategy/notification-strategy.factory.ts` `24-26`

`Frontend/STUDER/src/app/features/notifications/strategy/user-notification.strategy.ts` `9-11`